

Table of Contents

Git Version Control Document	2
Game Tournament Platform Database	2
Table of Contents	2
1. Project Summary	2
2. Technology Stack.....	3
3. Milestone Development Log	3
Milestone 1 â€” Requirements & Initial Design	3
Milestone 2 â€” Normalization (2NF â†’ 3NF)	3
Milestone 3 â€” Dataset Preprocessing	4
Milestone 4 â€” Database DDL.....	4
Milestone 5 â€” Data Population (DML)	4
Viva Preparation â€” Final Package	4
4. Git Commit Timeline	5
5. Repository File Structure	6
6. Schema Overview	7
Core Gameplay.....	7
Tournaments	8
Social.....	8
Communication.....	8
Analytics	8
Entity Relationship Diagram	9
7. Key Design Decisions	9
8. Known Issues Resolved	10
9. Validation Results	10
Row Counts (all 16 tables)	10
Integrity Checks	11
10. How to Run the Project	11
Prerequisites	11
Step 1 â€” Stage the CSV data.....	11
Step 2 â€” Run the master script	12
What the script does (in order)	12

Git Version Control Document

Game Tournament Platform Database

Course: Database Systems Laboratory **Project:** Game Tournament Platform Database **Database:** game_tournament_db (MySQL 8.0) **Repository:** GitHub ["branch main"](#) **Schema:** 16 tables, fully normalized to Third Normal Form (3NF)

Table of Contents

1. Project Summary
 2. Technology Stack
 3. Milestone Development Log
 4. Git Commit Timeline
 5. Repository File Structure
 6. Schema Overview
 7. Key Design Decisions
 8. Known Issues Resolved
 9. Validation Results
 10. How to Run the Project
-

1. Project Summary

The **Game Tournament Platform Database** backs an online platform for organizing and playing competitive strategy and board game tournaments. The system manages player accounts, game definitions, individual matches and their moves, tournament structures and brackets, social features (friends, groups, chat, messaging, notifications), and a leaderboard for ranking.

The project was developed across **five milestones**, evolving from an initial requirements document and a 14-table draft design into a fully normalized **16-table schema in 3NF**, complete with a synthetic dataset, full DDL (constraints, indexes, triggers, views, stored procedures), DML population scripts, and a single **exam-ready master script** that builds and validates the entire database in one run.

All source code, schema scripts, datasets, and documentation are version-controlled in a single Git repository on the main branch. This document records the development history, design rationale, and the final verified state of the project.

2. Technology Stack

Layer	Technology
Database	MySQL 8.0 (utf8mb4 / utf8mb4_0900_ai_ci)
Schema design	Normalized to 3NF â€” 16 tables
Server-side logic	Stored procedures (11), triggers (14), views (12) in <code>master.sql</code>
Data loading	INSERT scripts and LOAD DATA INFILE (CSV source of truth)
Frontend (companion app)	React + TypeScript, Vite
Realtime / backend services	Firebase
Version control	Git + GitHub (branch main)
Tooling	MySQL Workbench, VS Code

3. Milestone Development Log

Milestone 1 â€” Requirements & Initial Design

- Defined project scope: an online tournament platform for strategy and board games.
- Authored the **requirements document** and the **initial Entity Relationship Diagram (ERD)**.
- Produced a first **14-table design**.
- Known limitations of this draft (later corrected in M2):
 - GAME table hardcoded P1_ID / P2_ID columns (limited to two players).
 - Moves were stored as a single TEXT column (violating 1NF).
- **Key files:** milestone 1/Requirements.docx, milestone 1/main_design.md, milestone 1/mysql_setup.sql (14-table INSERT-based setup), milestone 1/complete_setup_csv.sql (16-table CSV-based setup).

Milestone 2 â€” Normalization (2NF â†’ 3NF)

- Applied **Second Normal Form (2NF)**: removed partial dependencies on composite keys.
- Applied **Third Normal Form (3NF)**: removed transitive dependencies.
- **Redesign decisions:**
 - Replaced hardcoded P1_ID / P2_ID with the **game_player junction table**, enabling a flexible N-player count per game.
 - Replaced the moves TEXT column with the **game_move table** â€” one move per row (1NF compliance, fully queryable move history).
- Produced the normalized ERD and a written justification of every normalization step.

- **Key files:** milestone 2/3NF_ERD_Table_Justification.md, milestone 2/final_normalized_erd.png.

Milestone 3 – Dataset Preprocessing

- Generated a **synthetic dataset** covering all 16 tables: 100 players, 50 game types, 50 structures, 60 tournaments, 100 participants, 50 games, 100 game_player, 100 game_move, 100 game_history, 50 groups, 100 group_list, 80 friends, 100 leaderboard, 100 notification, 50 chat, 100 message.
- **Validated** the dataset for referential integrity, absence of NULL violations, and absence of orphan (dangling foreign key) records.
- Produced **16 CSV files** plus a validation report.
- **Key files:** milestone 3/milestone3_dataset_preprocessing/ (16 CSVs + VALIDATION_REPORT.md).

Milestone 4 – Database DDL

- Authored the full **16-table CREATE TABLE script** with **named constraints** (primary keys, foreign keys, unique keys, check constraints where permitted).
- Added **indexes** on all foreign-key columns and frequently queried columns.
- Implemented server-side logic. The advanced SQL modules under new folder/sql/ddl/ define **12 stored procedures, 18 triggers, and 17 views**; the viva master.sql consolidates a curated, self-contained subset of **11 procedures, 14 triggers, and 12 views**.
- **Key file:** milestone 4/milestone4_database_setup_ddl/milestone4_ddl.sql.

Milestone 5 – Data Population (DML)

- INSERT-based population script for all 16 tables.
- Demonstration of **UPDATE and DELETE** operations.
- **Validation queries:** row counts, NULL checks, foreign-key orphan checks.
- A **LOAD DATA INFILE template** as a CSV-based population alternative.
- **Key files:** milestone 5/milestone5_data_population_dml/ (01_insert_data.sql, 02_update_delete_demo.sql, 03_validation_queries.sql, 04_validation_output_commented.sql, optional_load_data_infile_template.sql, and the csv/ folder holding the 16 source-of-truth CSV files).

Viva Preparation – Final Package

- **viva/master.sql** – a single combined script that runs top to bottom in MySQL Workbench and performs, in order:
 1. Database creation
 2. Drop tables (reverse FK order)
 3. Create tables (16 tables)

4. Indexes
 5. Load data (LOAD DATA INFILE from the MySQL Uploads folder)
 6. Triggers (14 " data integrity)
 7. Views (12 " reports & analytics)
 8. Stored procedures (11 " business logic)
 9. UPDATE / DELETE demo
 10. Validation queries (row counts, NULL checks, FK integrity)
 11. Sample queries (player, tournament, game, social, chat)
 12. Business analytics (advanced reporting)
- **viva/erd.md** " a Mermaid ERD of all 16 tables, with attributes and relationships.
 - **Verification:** the master script ran end to end successfully " all 16 tables loaded, **zero NULL violations, zero FK orphans, data quality 100%.**

4. Git Commit Timeline

Chronological history of the key commits on main:

Commit messages and dates below are transcribed verbatim from `git log` and can be cross-checked with `git log --format="%h %ad %s" --date=short`.

Hash	Date	Message
192b4ae	2026-05-17	m1
055cd91	2026-05-17	M2: Applied 2NF and 3NF normalization, updated ERD and schema
b2b6958	2026-05-17	m4
cc85971	2026-05-17	M5: Data populated validation queries added
277fb92	2026-05-17	Enhancement: Add comprehensive advanced features and documentation
c180f8c	2026-05-17	Create git control doc.docx
aedb322	2026-05-17	Update git control doc.docx
5446535	2026-05-17	m5
ed35b42	2026-05-17	Create 3NF_ERD_Table_Justification.docx

Hash	Date	Message
16b0171	2026-05-17	Update README.md
22e4059	2026-05-18	Create 3NF_ERD_Table_Justification.md
37f6d01	2026-05-18	feat: redesign UI + add all ERD entities + seed M5 dataset
dffc37	2026-05-18	fix: remove matchmaking countdown and add builtin bot pool fallback
1847657	2026-05-21	Create improvement.txt
c20973f	2026-05-21	Create settings.local.json
b00f740	2026-05-28	Update settings.local.json

Reading the history: the project scaffold (Vite/TypeScript config, README) was committed in early May. The bulk of the database work landed on **2026-05-17** – the milestone scaffolding (m1, m4, m5), normalization (M2, 055cd91), data population and validation (M5, cc85971), and the advanced SQL feature set (277fb92 – procedures, triggers, views, analytics). The 2026-05-18 commits add the Markdown normalization justification and the companion-app UI redesign with the seeded dataset; later commits cover bot-pool tuning and configuration housekeeping. The table above lists the key commits; the full git log contains additional intermediate and housekeeping commits (renames, .docx cleanups).

5. Repository File Structure

```
Game-Database-Project/
  app/                                # React/TypeScript frontend (Firebase)
  ,  src/
  ,  ,  App.tsx, main.tsx
  ,  ,  pages/                    # LandingPage, GameRoom, Lobby, P
  rofile, etc.
  ,  ,  components/              # SuperBoard, SubBoard, ChatBox,
  etc.
  ,  ,  hooks/                   # useGameState, useSocket
  ,  ,  firebase.ts
  ,  ,  package.json
  ,  ,  README.md
  ,  milestone 1/                # Requirements + initial design
```

```

â”,   â”œâ”€â”€ Requirements.docx
â”,   â”œâ”€â”€ main_design.md
â”,   â”œâ”€â”€ mysql_setup.sql           # 14-table INSERT-based setup
â”,   â””â”€â”€ complete_setup_csv.sql    # 16-table CSV-based setup
â”œâ”€â”€ milestone 2/                  # Normalization
â”,   â”œâ”€â”€ 3NF_ERD_Table_Justification.md
â”,   â””â”€â”€ final_normalized_erd.png
â”œâ”€â”€ milestone 3/                  # Dataset
â”,   â””â”€â”€ milestone3_dataset_preprocessing/
â”,   â”œâ”€â”€ 16x CSV files
â”,   â””â”€â”€ VALIDATION_REPORT.md
â”œâ”€â”€ milestone 4/                  # DDL
â”,   â””â”€â”€ milestone4_database_setup_ddl/
â”,   â”œâ”€â”€ milestone4_ddl.sql
â”,   â””â”€â”€ README.md
â”œâ”€â”€ milestone 5/                  # DML
â”,   â””â”€â”€ milestone5_data_population_dml/
â”,   â”œâ”€â”€ 01_insert_data.sql
â”,   â”œâ”€â”€ 02_update_delete_demo.sql
â”,   â”œâ”€â”€ 03_validation_queries.sql
â”,   â”œâ”€â”€ 04_validation_output_commented.sql
â”,   â”œâ”€â”€ optional_load_data_infile_template.sql
â”,   â””â”€â”€ csv/                      # 16 CSV files (source of truth)
â”œâ”€â”€ new folder/sql/              # Advanced SQL
â”,   â”œâ”€â”€ ddl/procedures.sql         # 12 stored procedures
â”,   â”œâ”€â”€ ddl/triggers.sql          # 18 triggers
â”,   â”œâ”€â”€ ddl/views.sql             # 17 views
â”,   â”œâ”€â”€ dml/                      # population scripts (mirror of milestone 5)
â”,   â”œâ”€â”€ analytics/business_analytics.sql
â”,   â”œâ”€â”€ examples/sample_queries.sql
â”,   â”œâ”€â”€ tests/test_suite.sql
â”,   â””â”€â”€ utilities/backup_restore.sql
â””â”€â”€ viva/                        # Final exam-ready package
    â”œâ”€â”€ master.sql                # COMPLETE: DDL + data + triggers + views + procs + queries
    â”œâ”€â”€ erd.md                    # Mermaid ERD (16 tables)
    â””â”€â”€ git_control.md            # This document

```

6. Schema Overview

The schema comprises **16 tables**, grouped by functional area:

Core Gameplay

Table	Purpose
player	Player accounts â€” credentials, ELO rank, profile

Table	Purpose
	metadata.
gametype	Definitions of supported games (rules, category, player count).
game	A single match instance, tied to a game type and (optionally) a tournament.
game_player	Junction table linking players to a game (supports N players per game).
game_move	One move per row for a game â€” full, queryable move history.
game_history	Completed-game records: outcome, duration, archival data.

Tournaments

Table	Purpose
structure	Reusable tournament format definitions (e.g. single-elimination, round-robin).
tournament	A tournament instance referencing a structure.
participant	Registration of a player into a tournament.

Social

Table	Purpose
friends	Friendship pairs between players (ordered PID1 < PID2).
group_t	Player groups / clans.
group_list	Membership junction linking players to groups.

Communication

Table	Purpose
chat	A chat channel / conversation between players.
message	Individual messages, with self-referencing threaded replies.
notification	System and social notifications addressed to players.

Analytics

Table	Purpose
leaderboard	Denormalized snapshot of player ranks for fast read queries.

8. Known Issues Resolved

During DDL development under **MySQL 8.0**, four CHECK constraints triggered **Error 3823**: “Column ‘...’ cannot be used in a check constraint ...: needed in a foreign key constraint ... referential action.” MySQL 8 forbids a CHECK constraint on any column that participates in a foreign key’s referential action.

Each was resolved by relocating the validation logic out of the CHECK constraint:

Removed CHECK constraint	Original intent	Resolution
chk_group_list_not_self_invited	Prevent a player inviting themselves to a group.	Moved logic to a trigger .
chk_friends_distinct_players	Prevent self-friendship (PID1 = PID2).	Moved to trigger trg_friends_no_self_friendship.
chk_chat_distinct_players	Prevent a chat between a player and themselves.	Enforced in the application layer .
chk_message_not_reply_to_self	Prevent a message replying to itself.	Enforced in the application layer .

Outcome: the schema compiles cleanly on MySQL 8.0 with no loss of integrity – every rule the removed constraints enforced is preserved, either by a trigger or by the application layer.

9. Validation Results

Results captured from the end-to-end run of viva/master.sql:

Row Counts (all 16 tables)

Table	Rows
player	100
gametype	50
structure	50
tournament	60
participant	100
game	50
game_player	100

Table	Rows
game_move	100
game_history	100
group_t	50
group_list	100
friends	80
leaderboard	100
notification	99
chat	50
message	100

Integrity Checks

- **NULL violations:** 0 across all 16 tables.
- **Foreign-key orphan records:** 0 across all 12 checked relationships.
- **Data quality score: 100.00%.**

Note on notification (99 vs 100): 100 notification rows are loaded from the CSV; the UPDATE / DELETE demonstration (02_update_delete_demo.sql) deletes one old notification record, leaving **99**. This is the expected, demonstrated outcome – not a data-loss defect.

The script ran successfully with all 16 tables loaded and every integrity check passing.

10. How to Run the Project

The entire database – schema, data, triggers, views, procedures, and validation – is built in a **single command** using the master script.

Prerequisites

- MySQL 8.0 (with MySQL Workbench or the mysql CLI).
- The secure_file_priv Uploads directory available for LOAD DATA INFILE.

Step 1 – Stage the CSV data

Copy all CSV files from:

milestone 5/milestone5_data_population_dml/csv/

into the MySQL Uploads folder (default on Windows):

C:\ProgramData\MySQL\MySQL Server 8.0\Uploads\

Step 2 – Run the master script

Option A – MySQL Workbench: open `viva/master.sql` and execute the whole script (it is ordered to run top to bottom).

Option B – Command line:

```
mysql -u root -p < viva/master.sql
```

What the script does (in order)

1. Creates the `game_tournament_db` database.
2. Drops existing tables in reverse foreign-key order.
3. Creates all 16 tables with constraints.
4. Builds indexes.
5. Loads data from the CSVs via `LOAD DATA INFILE`.
6. Installs 14 triggers.
7. Creates 12 views.
8. Creates 11 stored procedures.
9. Runs the `UPDATE / DELETE` demonstration.
10. Runs validation queries (row counts, NULL checks, FK integrity).
11. Runs sample queries across all functional areas.
12. Runs business-analytics reporting queries.

On completion the validation section reports row counts, NULL checks, and FK integrity – confirming a **100% data-quality** load.

End of Git Version Control Document.